Roast & Co. Campaign Crew - Full System Documentation

Welcome to the full technical documentation for the Roast & Co. AI-Native Mini CRM!

This document explains the complete flow, tech stack, architecture, and features of the entire application, which is split into two parts:

1. **The CRM (Roastco Campaign Crew)**: The main Next.js application where brands manage customers and draft campaigns.
2. **The Channel Service**: An isolated Node.js microservice that acts as a mock delivery network (like Twilio or SendGrid).

Tech Stack

Main CRM

- **Framework**: Next.js 14 (https://nextjs.org/) (App Router, Server Components)
- **Database**: Supabase (https://supabase.com/) (PostgreSQL)
- **AI/LLM**: Google Gemini API (gemini-2.5-flash) via the AI SDK
- **Styling**: Tailwind CSS, Lucide React (Icons)
- **Analytics**: Recharts (Dynamic Funnel & Timeline visualization)
- **Language**: TypeScript

Channel Service

- **Server**: Express.js (https://expressjs.com/) (Node.js)
- **Real-time UI**: Socket.IO
- **Language**: JavaScript

Core Features

1. **AI Campaign Agent**: Instead of manually building audience filters or writing copy, users type their goals in plain English (e.g., *"Send a 20% off discount to VIP customers who have ordered more than 3 times"*). The AI uses Google Gemini to automatically generate SQL-like filters and craft personalized copy for SMS, WhatsApp, or Email.
2. **Decoupled Architecture**: The CRM and the Delivery Network are completely separated. The CRM fires payloads and relies entirely on asynchronous webhooks to update message statuses, preventing server lockups during high-volume sends.
3. **Live Webhook Funnel**: The Channel Service simulates network latency, opens, clicks, and bounces, firing webhooks back to the CRM in real-time.
4. **Mutually Exclusive Analytics**: A live dashboard visualizes the funnel using Recharts, breaking down sent messages into perfectly mutually exclusive buckets (Clicked, Opened, Delivered, Failed, Pending) alongside a 7-day performance timeline.

Architecture & Complete System Flow

Here is the exact step-by-step flow of how data moves through the system when a user interacts with the CRM:

Step 1: Data Ingestion (Customers)

Brands import their customer lists (via CSV or manual entry) containing names, emails, phone numbers, and total orders. This data is securely stored in the Supabase customers table and forms the foundation of the CRM's audience.

Step 2: Campaign Drafting (AI Segmentation)

1. The user opens the Chat Interface and types a prompt.
2. The **Gemini LLM** interprets the request.
3. The AI scans the intent and generates a JSON payload containing segment rules (e.g., orders >= 3), a goal, and personalized copy.
4. The CRM filters the customers table to find the audience that matches the AI's rules.
5. A drafted Campaign and its associated Messages are saved to the Supabase database in a pending state.

Step 3: Asynchronous Dispatch (The Handoff)

1. The user reviews the draft in the UI. They can change the delivery channel (SMS, WhatsApp, Email) with a single click, which updates the database.
2. When the user clicks **Send**, the CRM fires the messages to the external **Channel Service** via an HTTP POST request to http://localhost:4000/send.
3. The Channel Service receives the payload, places it in an in-memory array, and instantly returns a 202 Accepted.
4. The CRM updates the database status to sent and shows a success toast. The CRM's job is now done.

Step 4: Simulation & Webhooks (The Loop)

Over the next 5-30 seconds, the isolated Channel Service simulates network latency and delivery outcomes:

1. **Delivered**: Occurs 2-5 seconds after sending.
2. **Failed**: A randomized chance a message bounces (simulating a bad number/email).
3. **Opened**: Occurs 5-15 seconds after delivery.
4. **Clicked**: Occurs 10-25 seconds after opening.

Whenever one of these simulated events occurs, the Channel Service fires an **HTTP POST Webhook** back to the CRM (/api/receipt).

Step 5: Webhook Processing & Analytics

1. The CRM receives the webhook payload containing the messageId, status, and timestamp.
2. The CRM securely updates the exact message row in the Supabase database.
3. The CRM's Analytics Dashboard continuously polls the database.
4. The dashboard recalculates the mutually exclusive funnel metrics and instantly plots the new outcomes on the Recharts line graph and pie chart.

System Design: Handling Volume, Scale, & Failures

While this is a lightweight implementation for the take-home assignment, it is architected with enterprise-scale principles in mind. Here is how this architecture handles real-world scale:

- **Decoupled Delivery**: The CRM does not wait for messages to be delivered. It fires them and trusts the webhook to return the outcome. This prevents the CRM server from locking up or timing out during a 100,000-message campaign.
- **Handling High Volume Dispatch (Theoretical)**: In a production environment with millions of messages, the initial dispatch loop would be offloaded to a background worker (like **Inngest** or **BullMQ**) to chunk messages into batches of 500, respecting the API rate limits of downstream providers.
- **Webhook Reliability & Retries**: The webhook endpoint (/api/receipt) is designed to be idempotent. If the CRM server goes down temporarily, the webhooks aren't lost—a production event queue (like AWS SQS or RabbitMQ) would automatically retry the webhooks with exponential backoff until the CRM successfully processes them.
- **Stateless Delivery**: The Channel Service has no database. If it crashes, it forgets everything. It relies entirely on the CRM to be the persistent source of truth, enforcing a strict unidirectional data flow.

Environment Setup

If you need to run this system from scratch:

1. Supabase (Database)

You need a Supabase project with three tables:

- customers (id, name, email, phone, total_orders)
- campaigns (id, name, goal, segment_filters, status, channel)
- messages (id, campaign_id, customer_id, content, channel, status, sent_at, delivered_at, opened_at, clicked_at)

2. CRM Setup

```
cd roastco-campaign-crew
npm install
```

Create a .env.local file:

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key
GOOGLE_GENERATIVE_AI_API_KEY=your_gemini_api_key
CHANNEL_SERVICE_URL=http://localhost:4000
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Run the CRM: npm run dev

3. Channel Service Setup

```
cd ../channel-service
npm install
```

Run the service: node index.js

Built for the Xeno Engineering Take-Home Assignment.